

Stichai — Directional Fill Session Handoff (Jun 15)

Paste first in any new session that continues the directional-fill work.

TL;DR

A flow-following ("hand digitizer") fill engine was built and **validated** on elongated strip shapes^{**} (hair locks, limb/cloth segments). It is wired into ``vectorize.js`` behind a flag, with **automatic scanline fallback**^{**} so output is **never worse than before**^{**}. It currently activates **ONLY** on clearly elongated regions ($\text{aspect} \geq 3:1$) where it is proven clean; everything else uses the existing scanline. The hard open problem is making it clean on **blob-like regions**^{**} (faces, torsos), which have an interior flow singularity that tangles the rows.

Files in this bundle

- ``directional-fill.js`` → ``lib/directional-fill.js`` (NEW module, 469 lines)
- ``vectorize.js`` → ``lib/vectorize.js`` (adds require + flag + fill gate at ~line 790)

Both pass ``node --check`` and ``require()`` loads clean. Deploy = drop both in ``lib/``, ``node --check`` both, commit, push. **Test after deploy**^{**}: the engine should behave exactly as before on faces/torsos (scanline), and use directional fill on long thin regions. Watch the EVP geometry on hair/strands for curved rows.

To disable instantly without redeploy: set env ``STICHAIDIRECTIONALFILL=0`` on Railway.

What works (VALIDATED with renders this session)

1. **Chamfer distance transform**^{**} (JS, two-pass) — matches scipy EDT to ~0.07px; fill-angle error ~4°. Solid.
2. **Flow field**^{**} — stitch direction = perpendicular to $\text{grad}(\text{distance})$ = along form.
Validated: on a curved tube it gives +58°/0°/-63° across the bend (follows curve).
3. **Across-coordinate U**^{**} — multi-source Dijkstra from the entry edge (NOT a single corner — that was a bug, fixed). Gives true distance-across-the-form.
4. **Along-coordinate V**^{**} — Dijkstra weighting along-flow steps.
5. **Row extraction by (U-band × V-column) bucket-AVERAGING**^{**} — THIS was the key breakthrough. Do NOT go back to "marching/following" iso-lines; every follow-based method tangled. Averaging cells per (band,column) gives clean parallel curved rows.
6. **Boustrophedon serpentine**^{**} using U-band adjacency.
7. **Elongation gate**^{**} (PCA eigenvalue aspect ratio) — only strips qualify.

Proven-clean render: a gently curved hair-lock strip fills with rows following the curve, 0 internal jumps. (Single quarter-circle arm also fills clean at lower density.)

What does NOT work yet (the real open problems)

1. **Blob/round regions tangle**^{**} An oval (face/torso) has a flow SINGULARITY at its

center (distance-to-boundary peaks $\rightarrow$ gradient $\approx 0 \rightarrow$ direction undefined). Rows scramble around it. This is THE core unsolved issue. Convexity is NOT the fix (oval is convex and still tangles).

2. **Fold-back shapes (arcs/U/S)** make the (U,V) coordinate non-injective (two points share U,V). A fold-split pre-pass was prototyped (`splitIfFolded``, still in the file) but recombining the two halves cleanly is unsolved $\rightarrow$ those shapes currently decline.
3. **No single global shape metric separates "works" from "tangles."** Tried & failed: fold-detection (angle spread), convexity (solidity), elongation. The curved arm (works, elong 1.94) and oval (tangles, elong 1.40) overlap on every axis. The TRUE separator is "singularity-free flow," which needs detecting the singularity itself, not a shape proxy. **This is the next thing to build.**

Current gate (conservative, safe)

In `directionalFill()`:

...

if (elongation(outer) < 3.0) return []; // only clearly elongated strips

...

This excludes the murky-middle curved arm too (plays safe). It activates on genuine long thin regions. Tune UP for safety, DOWN (toward ~ 2.0) only after solving the singularity problem so blobs don't leak through.

Suggested next-session order

1. **Singularity detection**: find interior cells where $|\text{grad}(\text{distance})| \approx 0$ AND distance is locally maximal (the medial peak). If a region has an isolated singularity $\rightarrow$ it's blob-like $\rightarrow$ either (a) decline to scanline, or (b) fill radially around the singularity (a different fill mode). Solving this lets directional fill safely activate on faces/torsos — the high-value case.
2. **Multi-region split at singularities/folds**: split where flow reverses, fill each monotonic sub-region, connect with a TRIM (cut+restart) not a continuous travel.
3. Lower the elongation gate once 1–2 are solid.
4. Underlay tuning to match the pro benchmark (see below).

Pro-file benchmark (from zambi_3_8_in.DST, decoded this session)

- Density **3.9 st/mm²** (yours ≈ 2.0) — pro is $\sim 2\times$ denser. Density is a bigger lever than we treated it. Consider pRow 0.3 $\rightarrow$ 0.22–0.25 as a separate quick win.
- Stitch length range p5=0.81mm ... max=10mm (wider than your 3.8 cap; pro uses long stitches in open fills, short in detail).
- Curved fills with dir-spread 35–62° per region — confirms hand digitizers vary stitch direction within a region. That's what directional fill reproduces.
- A clean pro DST STILL looks busy in EVP up close (stray detail lines, rough edges).

EVP geometry is real (positions are exact) but thread-width coverage is NOT simulated: 0.2mm-spaced rows look gapped in EVP yet thread ($\sim 0.4\text{mm}$) covers them on fabric. Some "streaks" are render artifacts; a few bold isolated cross-lines are real travels.

Hard-won lessons (don't repeat these)

- Bucket-AVERAGING beats iso-line FOLLOWING for row extraction. (5 follow attempts failed.)
- Multi-source edge seeding for U, not single-corner (corner gives geodesic-from-point, which wraps around curves and scrambles bands).
- Verify every gate with a RENDER. Every "this metric will work" claim this session was wrong until rendered. fold-detect, convexity, elongation each looked right and weren't.
- The engine's other proven fixes from prior sessions remain intact and should NOT be touched: dead-density fix (pRow via tatamiRow), travelCut 8mm, Gemini pro-image retry, face-simplify, outline dilation 1px (patch A). Patch B (in-fill bridge) was reverted (streaks were mostly render artifact).

Status line

Directional fill: KERNEL WORKS on strips (validated), WIRED safely (scanline fallback), ACTIVATES narrowly ($\text{elong} \geq 3:1$). Open: singularity handling for blobs (faces/torsos) — that's the unlock for broad activation, and it's a real algorithm to build next.